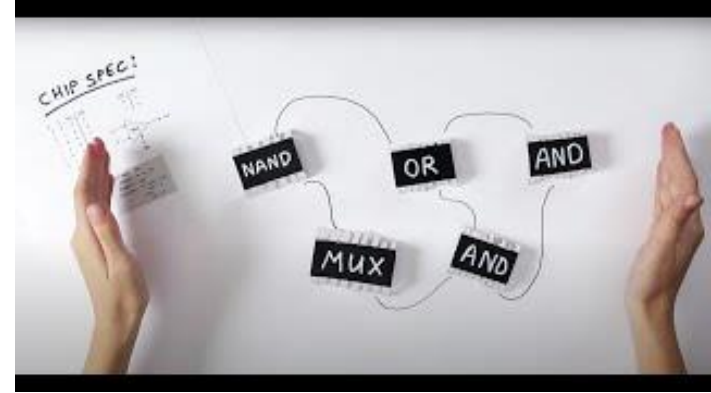




CSc 360

Operating Systems

Processes

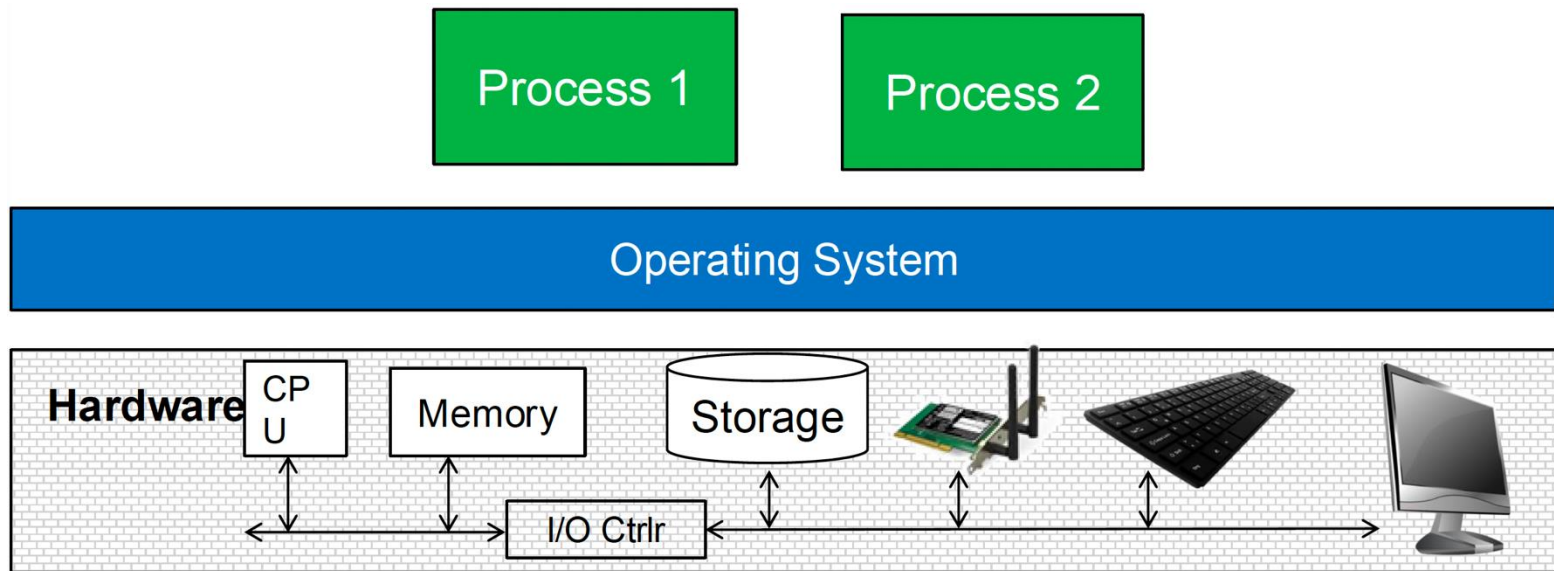
Wenjun Yang

Fall 2025

Recall: Topic Breakdown (Course Objective)

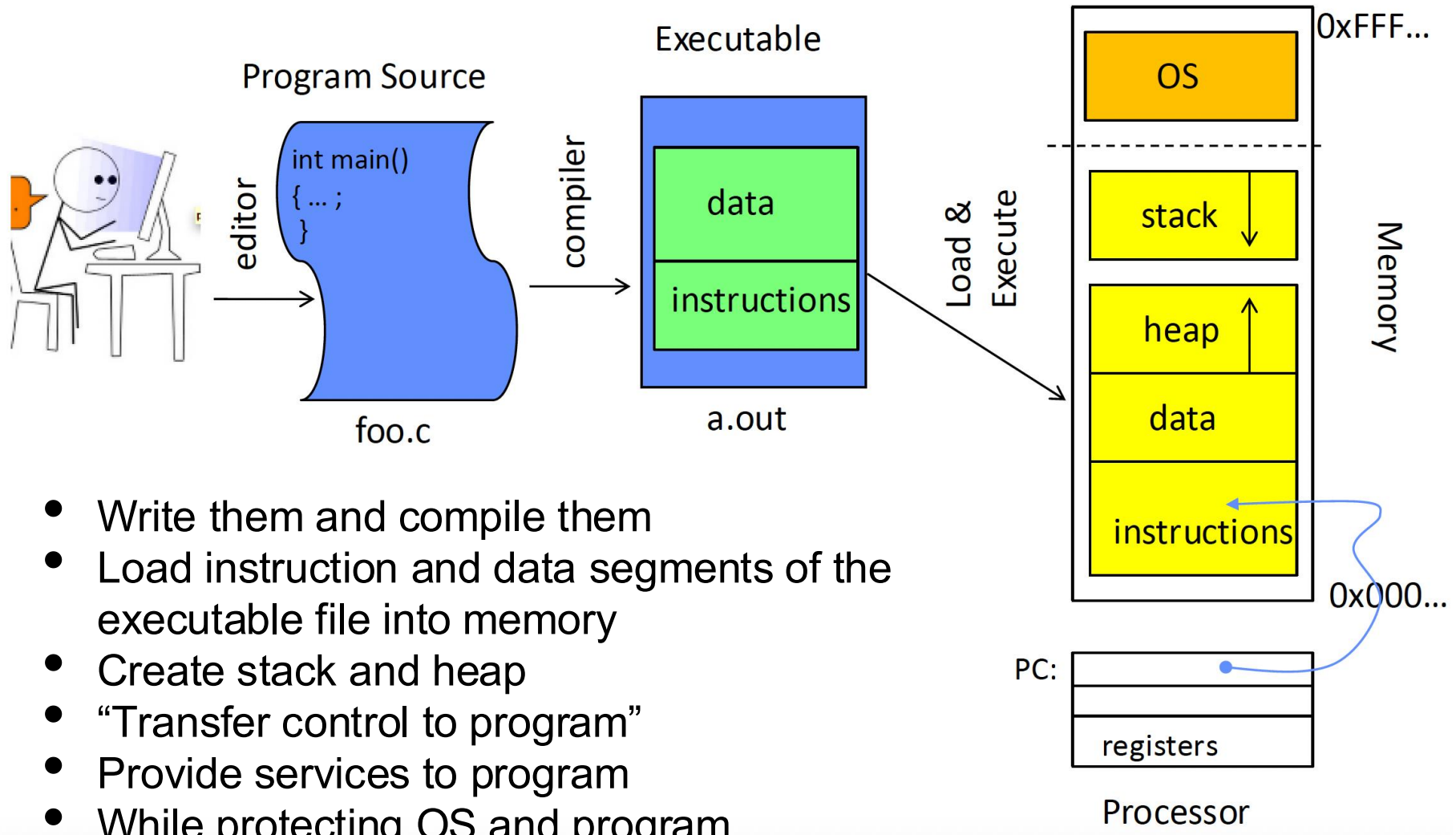
- virtualizing the CPU (**process**):
process, thread, scheduling, synch
- virtualizing memory (**memory**):
memory management, virtual memory
- **storage**: file systems, I/O systems

Process: an active instance of a program



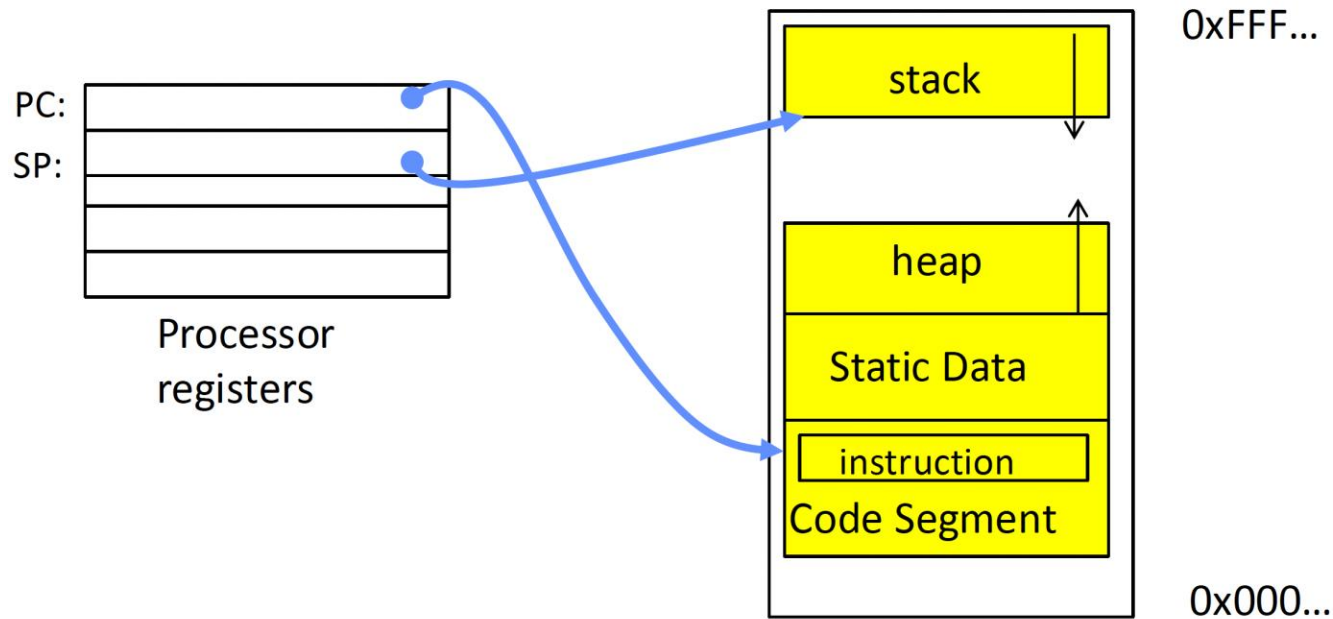
- OS virtualizes hardware to processes (applications) to provide the illusion that each process uses its own machine; also provides illusion of infinite memory and processor

OS Bottom Line: Run Programs



- Write them and compile them
- Load instruction and data segments of the executable file into memory
- Create stack and heap
- “Transfer control to program”
- Provide services to program
- While protecting OS and program

Address Space



- What's in the code segment? Static data segment?
- What's in the stack segment?
 - How is it allocated? How big is it?
- What's in the heap segment?
 - How is it allocated? How big?

Example

- Stack?
- Heap?
- Data?
- ...

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;

void stack_function() {
    int stack_var_in_func = 30;
    printf("Address of stack_var_in_func: %p\n", (void*)&stack_var_in_func);
}

int main() {
    int stack_var = 20;

    int *heap_var = (int*)malloc(sizeof(int));
    if (heap_var == NULL) {
        fprintf(stderr, "Failed to allocate memory on the heap\n");
        return 1;
    }
    *heap_var = 40; // Storing a value in the heap memory.

    // The programmer is responsible for freeing heap memory.
    free(heap_var);

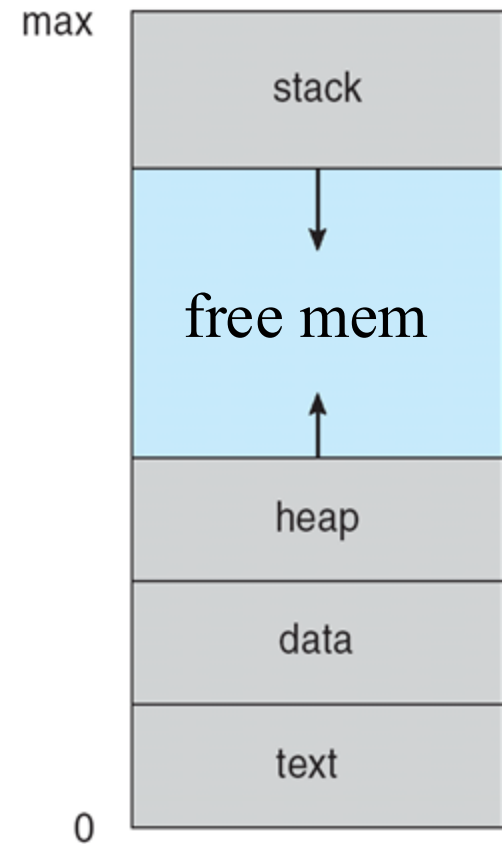
    return 0;
}
```

Diagram illustrating memory allocation locations for different variables in the code:

- (1) Points to `global_var = 10;` (Global variable, Data segment).
- (2) Points to `stack_var = 20;` (Local variable, Stack).
- (3) Points to `heap_var = (int*)malloc(sizeof(int));` (Heap-allocated memory, Heap).

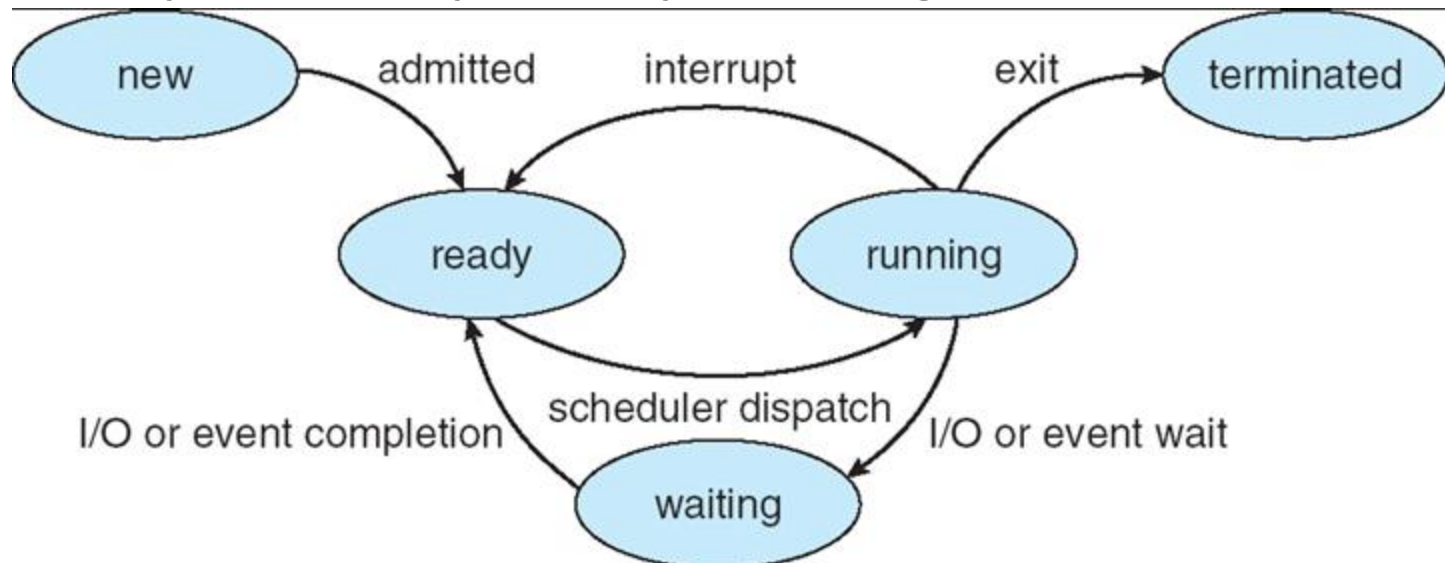
Processes

- Process: a program in execution
- Program: passive entity
 - static binary file on storage
 - e.g., `gcc -o hello hello.c`
- Process: active entity; resource allocated!
 - `./hello`
 - text (code); data (static), stack, heap
 - process control block (PCB)



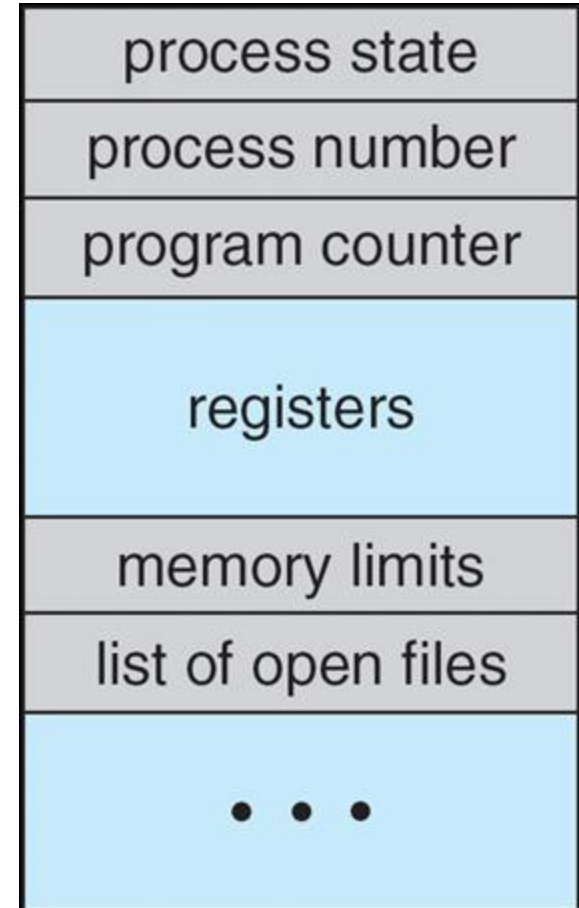
Process states

- E.g., one CPU (core)
 - one running process at any time
 - maybe many ready/waiting processes



Process control blocks

- PCB: keep track processes
 - state: ready/running, etc
 - CPU
 - PC, registers, priority, etc
 - memory
 - memory control information
 - I/O
 - e.g., list of opened files
 - accounting



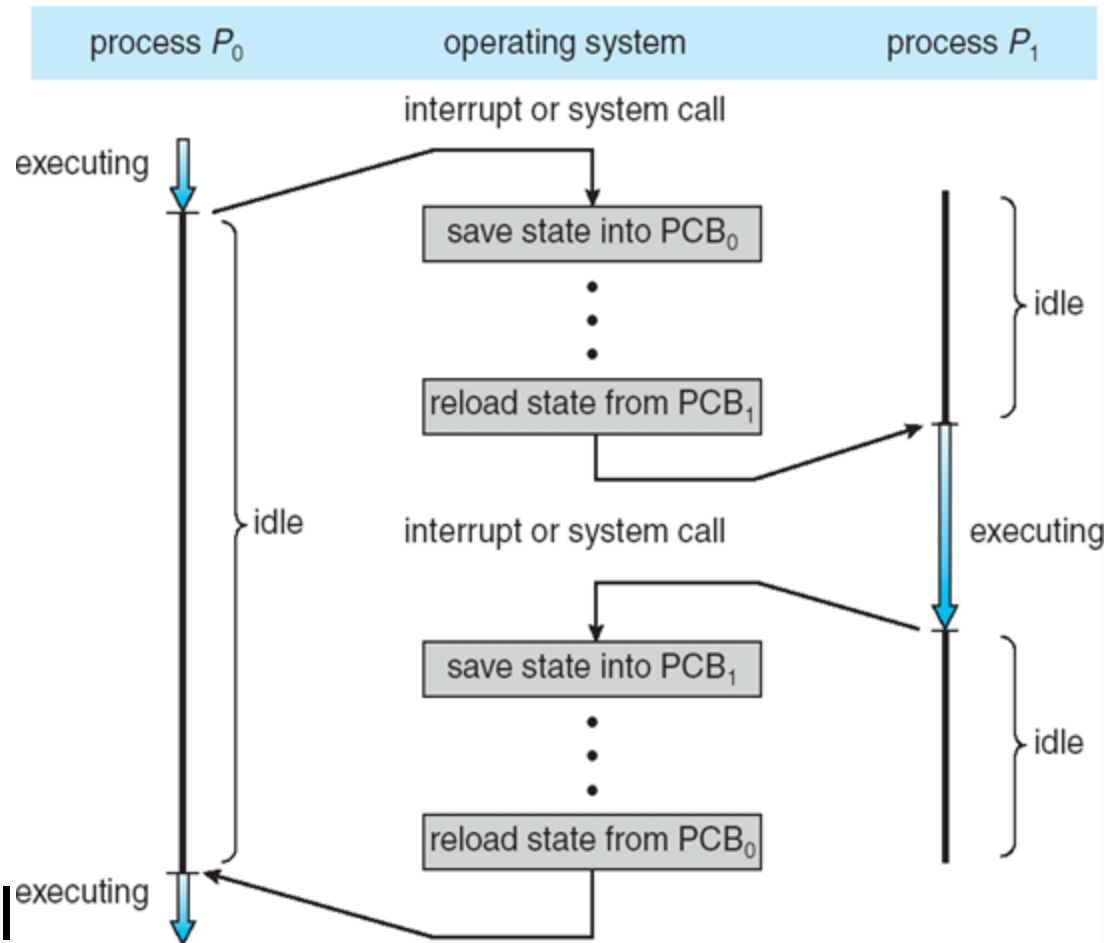
Context switching

- Context switch

- save states
- restore states

- When

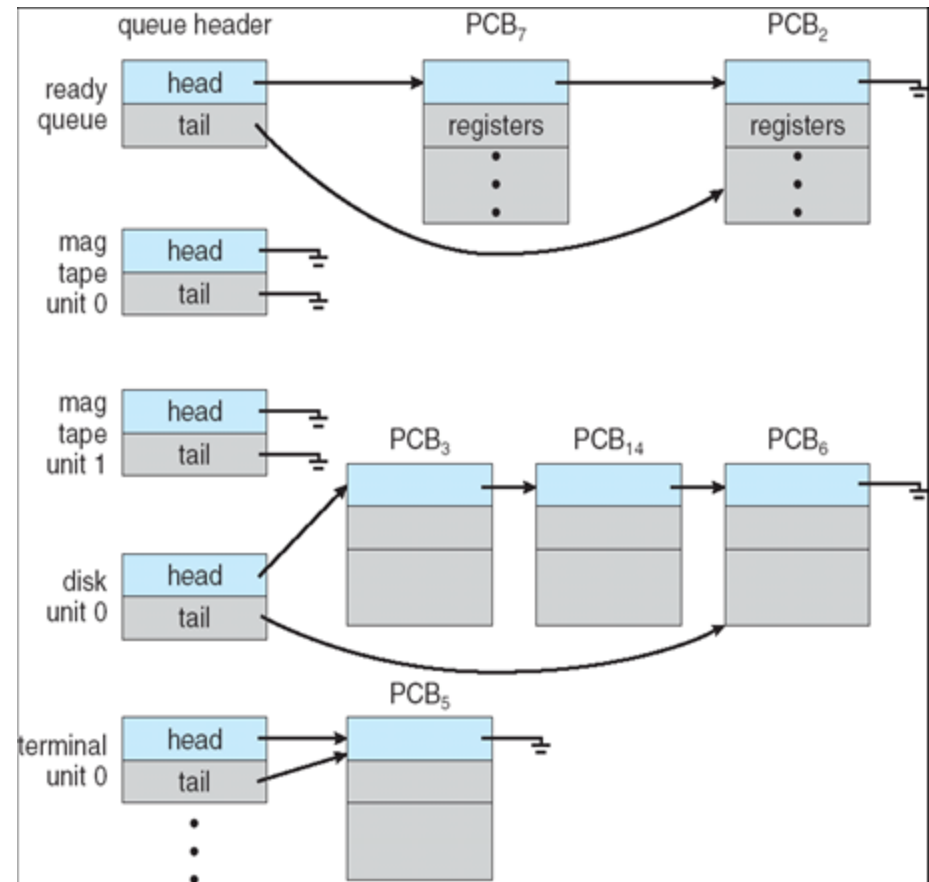
- timer
- I/O, memory
- trap
- waiting sys call



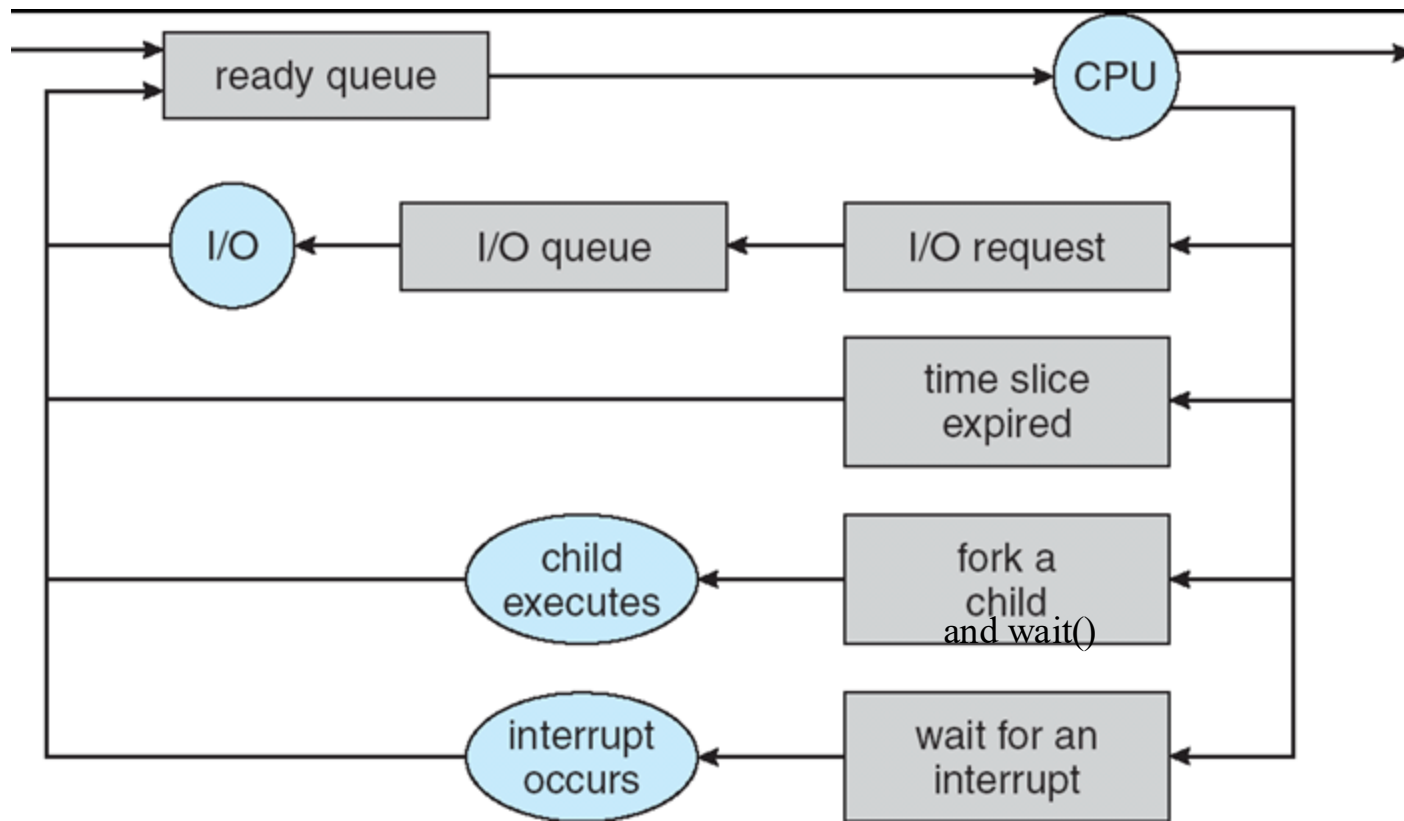
when more than two processes

Process scheduling

- Multiprogramming
 - utilization
- Timesharing
 - interactive
- Scheduling queues
 - linked list
 - ready queue
 - I/O queue



Queuing system



Queuing scheduler

- Who's the next?
- Long-term scheduler
 - job scheduler (spooling)
 - get to the ready queue
 - CPU-intensive vs I/O intensive
- Short-term scheduler
 - CPU scheduler
 - frequency vs overhead

More on scheduling

- Medium-term scheduler
 - who is NOT the next
 - reduce the degree of multiprogramming
 - swap-in/out
- Scheduling algorithms
 - first-come-first-server, shortest-job-first, priority, round-robin, fair and weighted fair, ...
 - more in Chapter 5

This lecture so far

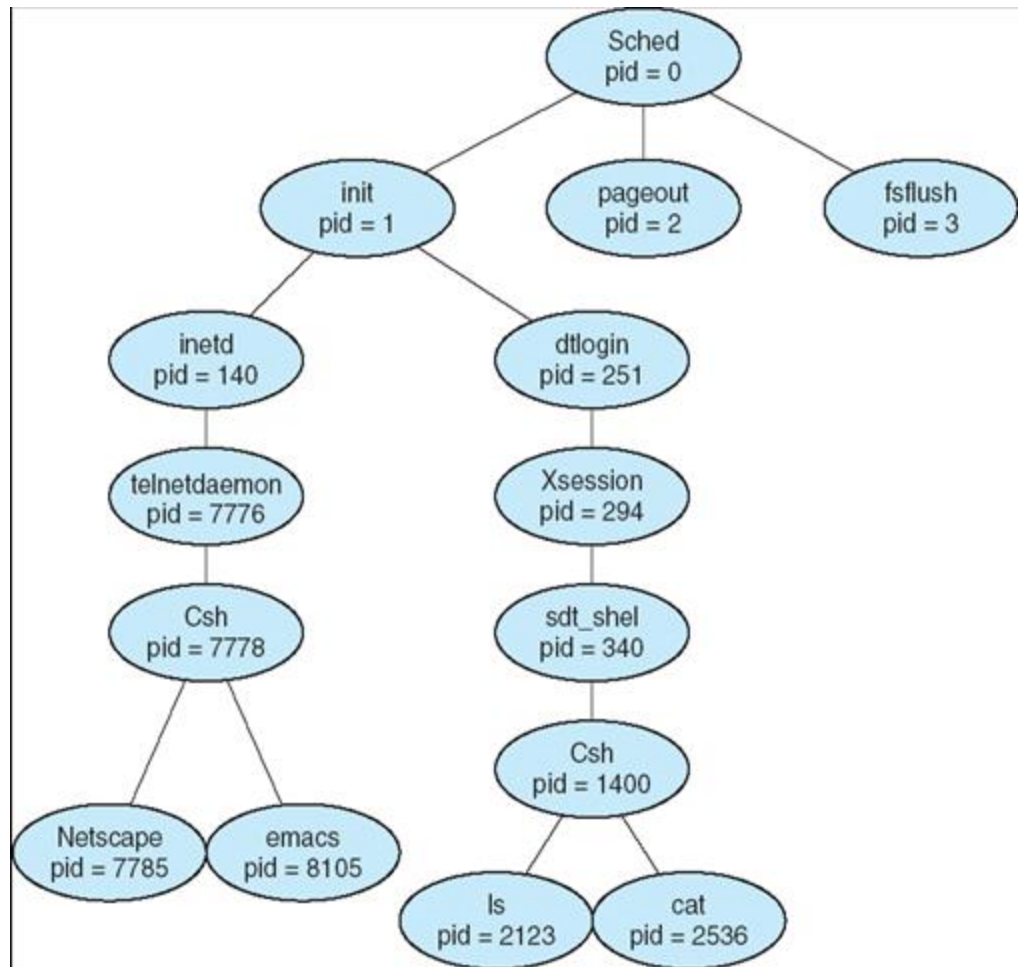
- Process and process scheduling
 - process vs program
 - process control block
 - context switch: what to save/restore
 - process scheduling
- Explore further
 - process status: `/bin/ps`
 - top CPU processes: `/usr/bin/top`

Process creation

- Creating processes
 - parent process: create child processes
 - child process: created by its parent process
- Process tree
 - recursive parent-child relationship; why tree?
 - `/usr/bin/pstree`
- Process ID (PID) and Parent PID (PPID)
 - usually nonnegative integer

Process tree

- sched (0)
 - init (1)
 - all user processes
 - pageout
 - fsflush
- file system



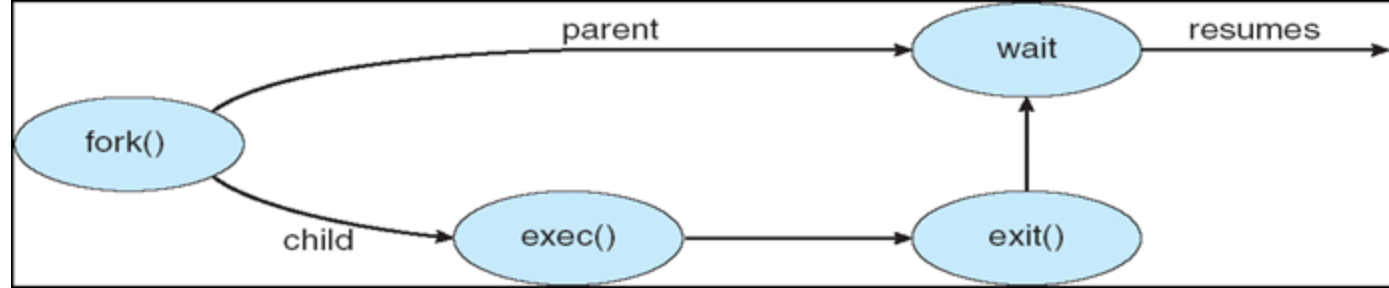
```
snmpd
sshd
  5*[sshd—sshd—sftp-server]
  sshd—sshd—bash—ssi
  sshd—sshd—bash—more
  sshd—sshd—csh—sftp-server
  sshd—sshd—bash
  sshd—sshd—bash—mutt
  sshd—sshd—tcsh—nano
  sshd—sshd—tcsh
syslogd
```

Parent vs child processes

- Process: running program + resources
- Resource sharing: possible approaches
 - all shared, or
 - some shared (e.g., read-only code), or
 - nothing shared*
- Process execution: possible approaches
 - parent waits until child finishes, or
 - parent and child run **concurrently***

fork(), exec*(), wait()

- Create a child process: fork()
 - return code < 0: error (in “parent” process)
 - return code = 0: you’re in child process
 - return code > 0: you’re in parent process
 - return code = child’s PID
- Child process: load a new program
 - exec*(): front-end for execve(file, arg, environ)
- Parent process: wait() and waitpid()



Example

```
int main()
{
    Pid_t pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) { /* child process */
        execvp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /*parent will wait for the child to complete*/
        wait (NULL);
        printf ("Child Complete");
        exit(0);
    }
}
```

CSc 360
* what if no wait()?

Process termination

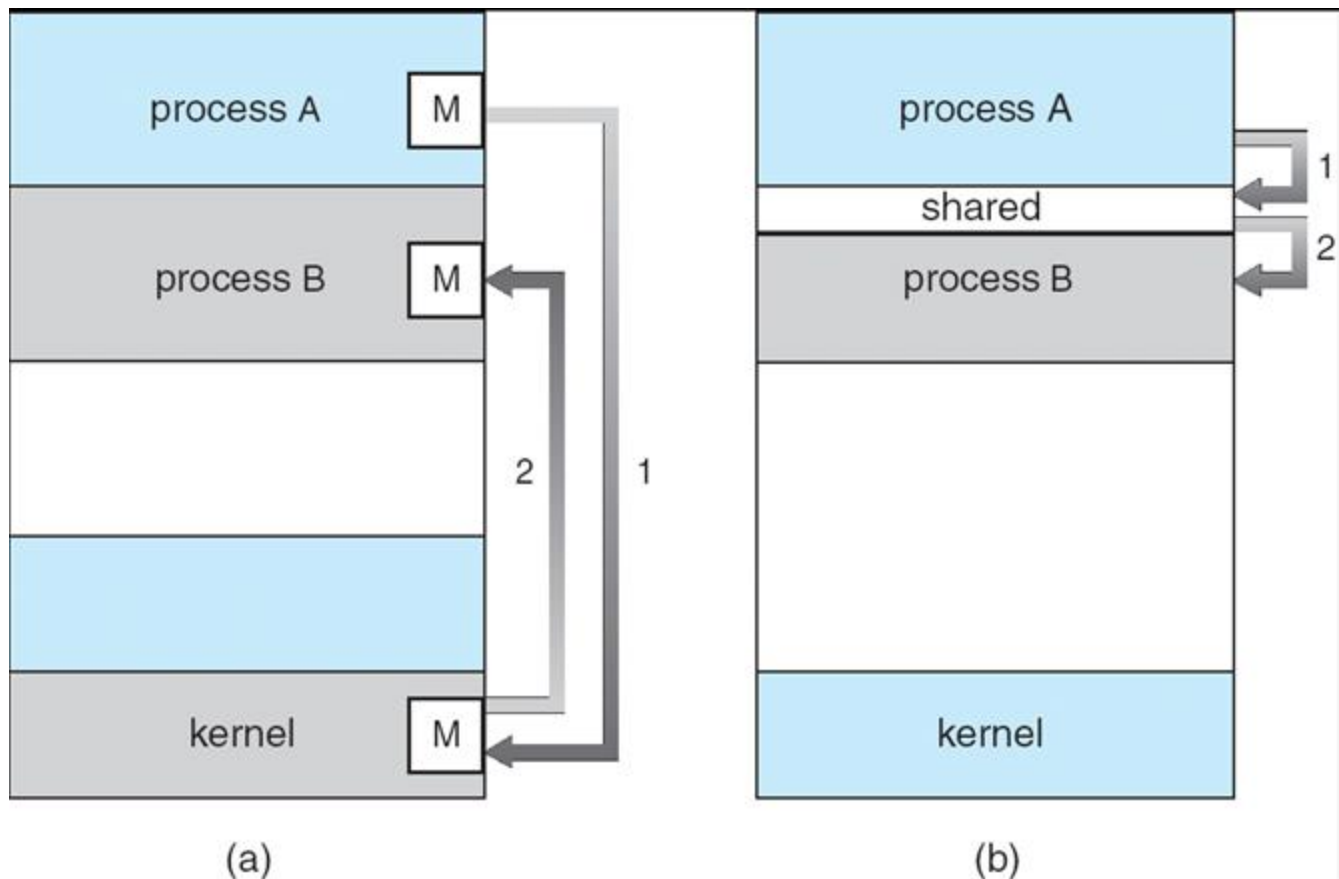
- Terminate itself: `exit()`
 - report status to parent process
 - release allocated resources
- Terminate child processes: `kill(pid, signal)`
 - actually send a signal to the child
 - child resource exceeded, child process no longer needed, and so on
 - parent is exiting
 - cascading termination, or find another parent

Process communication

- Independent process
 - standalone process
- Cooperating process
 - affected by or affecting other processes
 - sharing, parallel, modularity, convenience
- Process communication
 - shared memory
 - message passing

Message passing vs shared memory

- Overhead
- Protection



The 2nd half of this lecture

- Process operations
 - process creation
 - process tree
 - process termination
 - the need for inter-process communication
- Explore further
 - /bin/ps, /usr/bin/top, /usr/bin/pstree
 - how does a child process find its parent's PID?

Next lecture

- Inter-process communication
 - read OSC7 Chapter 3 (or OSC6 Chapter 4)